

# Enterprise Knowledge Systems

## Abstract

Organisations accumulate vast quantities of knowledge across documents, codebases, conversations and structured data, yet struggle to leverage it due to information silos, re-discovery costs and institutional knowledge loss. This paper presents an enterprise knowledge system, formalises the embedding pipeline and graph structures that underpin it, and grounds the discussion in an implemented codebase knowledge-graph pipeline. We describe the architecture, graph construction process and retrieval evaluation of that system, before identifying directions toward dynamic knowledge graphs. A practical example from financial markets illustrates the reasoning capabilities that motivate building a knowledge system.

## 1 Introduction

Organisations cannot efficiently leverage their accumulated knowledge due to several critical limitations. Knowledge silos isolate departmental expertise, preventing the spread of insights and duplicating effort across teams. Re-discovery costs compound as employees repeatedly search for previously found information without persistent memory. Institutional knowledge is lost when experts leave.

Consider the following scenarios:

- (i) A new company has been onboarded. An employee must sift through contracts, regulatory filings, org charts and past projects to prepare a summary—information that could be pre-connected in a knowledge system rather than manually reassembled each time.
- (ii) A client requests a structured product. A similar one was delivered for a different client, but the previous solution is scattered across email threads and shared drives.
- (iii) A developer modifies a shared library on which dozens of repositories depend, but only they understand which services are affected and how. The developer leaves. The dependency relationships exist in code but are not surfaced in any searchable form.

To address these challenges, we look to tools such as vector databases, retrieval-augmented generation (RAG) and knowledge graphs. Section 2 establishes the technical foundations, section 3 traces the knowledge spectrum from RAG to knowledge graphs, and section 4 describes our implemented system. Section 5 outlines forward-looking directions toward dynamic architectures, and section 6 illustrates the value of multi-hop reasoning through a practical example.

## 2 Technical Foundations

### 2.1 Preliminaries: Chunking, Embedding and Vector Storage

We assume familiarity with embedding models [1, 2] and vector similarity search. This section establishes notation and highlights design choices relevant to later sections.

**Chunking.** Documents must be split into segments before embedding. Strategies range from fixed-size windows to semantic splitting at paragraph or section boundaries and hierarchical nesting. For source code, chunking should preserve structural boundaries (functions, classes, modules) so that retrieved chunks contain complete logical units.

**Embedding.** Transformer-based encoders map discrete text to continuous vector space. Let  $\mathcal{X}$  denote the discrete text space,  $\mathcal{T}$  the token vocabulary, and  $\mathcal{V} := \mathbb{R}^d$  the embedding space with  $d \in \mathbb{Z}^+$ . Given an input  $\mathbf{x} \in \mathcal{X}^k$  of length  $k$ , a chunking strategy  $\chi$  partitions  $\mathbf{x}$  into  $m$  chunks of length at most  $L$  (the model’s context limit):

$$\chi_{\text{fixed}} : \mathcal{X}^k \rightarrow \{\mathcal{X}^L\}^m, \quad m = \left\lceil \frac{k}{L} \right\rceil, \quad (1)$$

$$\chi_{\text{semantic}} : \mathcal{X}^k \rightarrow \{\mathcal{X}^{\leq L}\}^m. \quad (2)$$

A tokenisation mapping  $\tau : \mathcal{X}^{\leq L} \rightarrow \mathcal{T}$  converts text into tokens. A learned function  $f : \mathcal{T}^{\leq L} \rightarrow \mathcal{V}$  produces the embedding. The encoder is their composition:

$$E = f \circ \tau, \quad E(\mathbf{x}) = \mathbf{v} \in \mathcal{V}. \quad (3)$$

Embeddings are typically  $L_2$ -normalised so that  $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|_2$  with  $\|\hat{\mathbf{v}}\|_2 = 1$ , yielding the full pipeline:

$$E(\mathbf{x}) = f(\tau(\chi(\mathbf{x}))) = \hat{\mathbf{v}} \in \mathcal{V}. \quad (4)$$

**Vector similarity and retrieval.** Vector databases store embeddings and handle retrieval. Given a query vector  $\mathbf{q}$  and a stored vector  $\mathbf{v}$ , similarity is typically measured by cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{v}) = \frac{\langle \mathbf{q}, \mathbf{v} \rangle}{\|\mathbf{q}\| \|\mathbf{v}\|}, \quad (5)$$

where  $\langle \mathbf{q}, \mathbf{v} \rangle$  denotes the inner product. At query time, retrieval returns the top- $k$  stored vectors with the highest  $\text{sim}(\mathbf{q}, \mathbf{v})$  values. If  $\mathbf{q}$  is also a  $L_2$ -normalised vector this reduces to  $\langle \mathbf{q}, \mathbf{v} \rangle$ .

## 2.2 Graph Databases

Graph databases extend vector storage by introducing relationship metadata and graph-traversal capabilities. Whilst vector databases capture semantic similarity between isolated chunks, graph databases represent structured relationships that enable multi-hop reasoning.

### Formal background

A *labelled property graph* [3] is defined as:

$$G = (\mathcal{E}, \mathcal{R}, \mathcal{F}), \quad (6)$$

where:

- $\mathcal{E}$  is the set of entities (nodes), each carrying a label from a label alphabet  $\Lambda$  and key-value properties,
- $\mathcal{R}$  is the set of relationship types,
- $\mathcal{F} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$  is the set of facts (directed edges), which may also carry properties.

## Triples and quadruplets

The atomic unit of a knowledge graph is a *triple* [4]:

$$(s, r, o), \quad s, o \in \mathcal{E}, r \in \mathcal{R}, \quad (7)$$

asserting that subject  $s$  is related to object  $o$  via relationship  $r$ . A *quadruplet* extends this with edge properties:

$$(s, r, o, p), \quad (8)$$

where  $p$  may carry a confidence score, weight, timestamp or other metadata.

## Graph traversal

Given a starting entity  $v_1$  (typically determined by a query match or a known entry point), a  $k$ -hop path through the graph is a sequence:

$$v_1 \xrightarrow{r_1} v_2 \xrightarrow{r_2} \dots \xrightarrow{r_k} v_{k+1}; \quad (v_i, r_i, v_{i+1}) \in \mathcal{F}, \quad (9)$$

## Building a graph

The processes for building a graph are:

1. **Entity extraction:** algorithms identify distinct entities and create corresponding nodes. Entity deduplication ensures uniqueness.
2. **Relationship inference:** connections between entities are determined from structural, semantic or co-occurrence signals, producing triples  $(s, r, o)$ .
3. **Quality assessment** (iterative): duplicate and low-confidence triples are removed, among other measures.

For code, deterministic parsers such as tree-sitter [5] extract entities reliably; for unstructured text, large language models are increasingly used to extract entities and relationships in a single pass [6].

# 3 The Knowledge Spectrum

## 3.1 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [7] grounds agent responses in external knowledge by retrieving relevant content and injecting it as context. Traditional RAG struggles when logically related documents use different vocabulary: their embeddings land far apart in vector space even though the underlying concepts are tightly connected. For example, consider an electric vehicle manufacturer’s quarterly earnings report and an announcement on lithium mining restrictions. These are highly related in the real world—EV manufacturer  $\rightarrow$  lithium dependency for battery production  $\rightarrow$  mining of lithium—yet their individual document embeddings would be very different, because they discuss distinct topics in distinct language.

## 3.2 Knowledge Graphs

A knowledge graph (KG) decomposes information into entities and relationships, making connections such as the EV-to-lithium chain possible (eq. (9)).

KGs can draw on both explicit sources (documents, structured data, conversations, user declarations) and implicit signals such as access patterns and dwell time.

Graph structures can be composed of layers, each sharing the same entity set but carrying distinct relationships:

$$G_l = (\mathcal{E}, \mathcal{R}_l, \mathcal{F}_l), \quad l \in \{1, \dots, n\}.$$

Slow-moving knowledge (corporate structures, legal entities) should sit on a different layer to fast-moving knowledge (current projects, news articles). Some layers need updating less frequently; some contain more sensitive information.

*Static* KGs are built from a fixed information sets. They remain constant across query sessions and must be rebuilt or patched offline to incorporate new information. A static graph rebuilt on a frequent schedule (e.g. weekly) can approximate freshness without the architectural complexity of a fully dynamic system—this is the approach taken in our implementation (section 4).

*Dynamic* KGs, by contrast, continuously evolve their structure through live information sources—user interaction, feedback loops, real-time data ingestion—and can update or create entities and relationships on-the-fly. We discuss dynamic KG directions further in section 5, after grounding the discussion in our implemented system.

## 4 Implementation

### 4.1 Construction

The weekly process instantiates a labelled property graph  $G = (\mathcal{E}, \mathcal{R}, \mathcal{F})$  (eq. (6)) and the embedding pipeline  $E$  (eq. (4)) over internal code repositories. Tree-sitter [5] extracts code entities across 13 languages and populates  $\mathcal{E}$ ,  $\mathcal{R}$  and  $\mathcal{F}$ . Entity deduplication is enforced via OpenCypher `MERGE` operations. The following label alphabet and relationship sets are used:

$$\Lambda_{\text{code}} = \{\text{Project, Package, Module, Class, Function, Method, Interface, Enum, Type, File, ExternalPackage}\},$$

$$\mathcal{R}_{\text{code}} = \{\text{CONTAINS\_*, DEFINES, IMPORTS, CALLS, INHERITS, OVERRIDES, IMPLEMENTS, DEPENDS\_ON\_EXTERNAL}\}.$$

Non-code files (documentation, configuration) are ingested as `File` nodes; shared structural nodes (`Project`, `Package`, `Folder`) tie code and documents on the same graph. Entities in  $\{\text{Class, Function, Method, Module, File}\}$  are embedded via OpenAI’s `text-embedding-3-large` ( $d = 3072$ ) model.

### 4.2 Evaluation

We evaluate retrieval quality as an *ablation study* using agents: the same 40-query set is run through four modes (`agent_only`, `agent_vdb`, `agent_graph`, `agent_vdb_graph`).

Queries are partitioned into four categories (10 semantic, 10 structural, 10 multi-hop, 10 negative control). All queries reference proprietary internal entities (class names, repository names, specific implementation details) that cannot be answered from LLM training data alone.

**Metrics.** We use two RAGAS [8] metrics that require no human-written reference answers: *context precision* (CP), measuring relevance of retrieved context to the query, and *faithfulness* (F), measuring whether the response is supported by retrieved context. Reference-free evaluation is appropriate here because the target codebases change frequently; fixed gold answers can become stale and penalise correct, up-to-date responses.

To score all modes (including `agent_only`, which has no retrieved context), we also compute an LLM-judge score (J): the mean of three dimensions: correctness, completeness and groundedness. The judge scores all mode responses for each query in one pass.

Each query is evaluated once per mode at temperature 0 with fixed evaluation and judge models. We report means over five runs. We record total latency, token counts, and the number of tool calls made by the agent. Table 1 reports aggregate and category-level results.

Table 1: Ablation results by mode and query category (5 runs).

| Category   | Mode                | CP   | F    | J    | Tool calls |
|------------|---------------------|------|------|------|------------|
| Overall    | Agent only          | —    | —    | 0.06 | 0.0        |
|            | Agent + VDB         | 0.70 | 0.70 | 0.45 | 8.6        |
|            | Agent + Graph       | 0.43 | 0.49 | 0.63 | 27.1       |
|            | Agent + VDB + Graph | 0.65 | 0.66 | 0.75 | 13.5       |
| Semantic   | Agent only          | —    | —    | 0.00 | 0.0        |
|            | Agent + VDB         | 0.74 | 0.83 | 0.59 | 11.2       |
|            | Agent + Graph       | 0.19 | 0.40 | 0.49 | 37.0       |
|            | Agent + VDB + Graph | 0.55 | 0.75 | 0.69 | 16.1       |
| Structural | Agent only          | —    | —    | 0.02 | 0.0        |
|            | Agent + VDB         | 0.53 | 0.58 | 0.21 | 10.0       |
|            | Agent + Graph       | 0.69 | 0.56 | 0.87 | 12.0       |
|            | Agent + VDB + Graph | 0.65 | 0.48 | 0.91 | 7.9        |
| Multi-hop  | Agent only          | —    | —    | 0.03 | 0.0        |
|            | Agent + VDB         | 0.64 | 0.63 | 0.38 | 9.4        |
|            | Agent + Graph       | 0.44 | 0.43 | 0.68 | 33.1       |
|            | Agent + VDB + Graph | 0.54 | 0.64 | 0.72 | 21.8       |
| Negative   | Agent only          | —    | —    | 0.18 | 0.0        |
|            | Agent + VDB         | 0.88 | 0.78 | 0.64 | 3.9        |
|            | Agent + Graph       | 0.42 | 0.56 | 0.49 | 26.5       |
|            | Agent + VDB + Graph | 0.85 | 0.78 | 0.70 | 8.2        |

Table 2: Latency and token cost by mode (1 run).

| Mode                | Total (s) | Input tok. | Output tok. | Total tok. |
|---------------------|-----------|------------|-------------|------------|
| Agent only          | 70.5      | 226        | 1817        | 2043       |
| Agent + VDB         | 24.1      | 163084     | 1474        | 164558     |
| Agent + Graph       | 19.3      | 129450     | 3959        | 133408     |
| Agent + VDB + Graph | 20.0      | 190447     | 2210        | 192657     |

### 4.3 Observations

Agent-only is a trivial control mode, performing as expected. VDB-only logically offers substantial improvements. A known limitation of CP is that it penalises larger contexts even when the additional information improves the answer. Judge scores (J) are the most robust metric.

**Graph contribution is category-specific.** The graph-only mode is strongest on structural queries, while VDB-based modes are stronger on semantic queries. Across all categories in this run, `agent_vdb_graph` gives the highest overall judge score.

**Negative controls.** Negative-control queries expose a limitation of top- $k$  retrieval: vector search still returns nearest neighbours even when no truly relevant evidence exists. This could

be mitigated by adding a similarity floor.

**Costs.** Latency is comparable between modes. Agent-only takes the longest because it doesn’t know how to respond to proprietary queries. Agents with tools have high input token counts as the queries are difficult, requiring multiple tool calls. However, output tokens are also comparable. Graph-only modes overuse the tool as they cannot surface raw code and rely on entity names. The high tool call counts reflect in the low CP scores.

**Limitations.** The evaluation runs queries in parallel, which could inflate latency figures due to server contention. We also do not separate tool retrieval from generation. The graph currently contains only source code and associated files. The architecture supports extension to business documents via the existing `File` node infrastructure and shared structural parents, moving the system toward a broader enterprise knowledge graph.

## 5 Future Directions

We describe a static system rebuilt weekly from a snapshot of active repositories. A natural extension is a *dynamic* knowledge graph that evolves continuously, balancing stability with adaptability. We outline three directions.

*Differential updates.* Rather than full rebuilds, the fact set can evolve incrementally:

$$\mathcal{F}_{t+1} = (\mathcal{F}_t \cup \Delta\mathcal{F}^+) \setminus \Delta\mathcal{F}^-, \quad (10)$$

where  $\Delta\mathcal{F}^+$  and  $\Delta\mathcal{F}^-$  are the facts added and removed between epochs.

*Feedback-driven weighting.* User interaction signals (explicit ratings, click-through, dwell time) can adapt edge weights online:

$$w(r) \leftarrow w(r) + \eta \cdot \text{feedback}(r), \quad (11)$$

strengthening frequently useful relationships and down-weighting noisy ones.

*Personal memory layers.* A user-specific weight function over the shared graph enables personalised retrieval without duplicating structure:

$$G_u = (\mathcal{E}, \mathcal{R}, \mathcal{F}, w_u). \quad (12)$$

These directions carry open challenges—quality degradation from feedback loops, privacy isolation of personal subgraphs, and scalability of weighted traversal—and remain active research areas [4]. The weekly-rebuild approach offers a pragmatic middle ground.

## 6 Practical Example

A practical use case for a knowledge system is detecting second-order market effects. Suppose a portfolio holds one or more of Companies A, B, and C.

**Query:** “Are any of my holdings likely to be mispriced given recent news?”

**Reasoning:** Daily article ingestion identifies a positive announcement from Company A. The graph also records that A is a major commercial partner of B, and that B is a supplier to C:

$$A \xrightarrow{\text{major customer of}} B \xrightarrow{\text{supplier to}} C.$$

If A’s announcement suggests stronger near-term demand, that may improve B’s revenue outlook. The possible impact on C is weaker but worth investigating.

**Response:** “Company B may be underpriced relative and Company C is worth further review because. . .”

Whilst this chain of thought is entirely possible for a human, the discovery and quantification of such relationships may be missed, especially as the number of hops increases. These chains can be executed at scale (thousands of companies) and simultaneously. Three compact reasoning dimensions matter most:

**Structural reasoning.** Similarity search alone may retrieve A-related news, but misses explicit multi-hop structure such as  $A \rightarrow B \rightarrow C$ .

**Semantic reasoning.** Relationship type changes the inference direction: partner edges can propagate positive signals, while competitor edges can invert them.

**Temporal reasoning.** A dynamic extension can reweight links as conditions change, so the same relationship chain can produce different recommendations at different times (section 5).

## 7 Conclusion

This paper has traced the knowledge spectrum from vector retrieval to entity-level knowledge graphs, presented a working codebase pipeline that implements static knowledge graph construction with dual vector/graph storage, and robust evaluation.

The implemented system demonstrates that even a static knowledge graph over source code enables queries that pure vector similarity cannot support. The architecture’s existing support for non-code files and shared structural nodes provides a natural path toward broader enterprise knowledge graphs that encompass business documents alongside code.

We have outlined future directions that extend the formalism toward dynamic architectures. The practical example from financial markets highlights advantages over human reasoning.

As managed graph services mature and construction tooling improves, the barrier to improved knowledge systems reduces.

## References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019.
- [2] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.
- [3] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys*, 40(1):1–39, 2008.
- [4] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, volume 26, 2013.
- [5] Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io/tree-sitter/>, 2018. Accessed: 2025-03-24.
- [6] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [8] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023.